LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 05 – RELASI ANTAR KELAS



Maysa Azhra Fauziyyah

103132430005

S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

5.1 Diagram Kelas

Diagram kelas dipakai buat menggambarkan kelas-kelas di dalam sistem beserta hubungan antar kelasnya. Satu kelas digambar sebagai kotak dengan 3 bagian: nama kelas, atribut, dan method. Access modifier ditulis pakai simbol: + untuk public, - untuk private, dan # untuk protected.

5.2 Hubungan Antar Kelas

Dalam OOP, kelas-kelas bisa punya hubungan satu sama lain. Jenisnya ada asosiasi, agregasi, komposisi, dan generalisasi/spesialisasi (yang terkait pewarisan).

5.2.1 Asosiasi

Asosiasi adalah hubungan antara dua kelas terpisah yang saling berkomunikasi lewat objek. Objek kelas lain cuma dipakai lewat parameter method dan nggak disimpan sebagai atribut. Contohnya Dosen yang memberi nilai ke Mahasiswa lewat method giveScore(Mahasiswa s, int nilai).

5.2.2 Agregasi

Agregasi adalah hubungan yang lebih kuat dari asosiasi, karena ada hubungan bagian (*part*) dan keseluruhan (*whole*). Meski begitu, kedua kelas tetap bisa dibuat sendiri-sendiri. Contohnya Jurusan yang punya atribut array Mahasiswa. Objek Mahasiswa dibuat di luar, lalu dimasukkan ke Jurusan.

5.2.3 Komposisi

Komposisi adalah bentuk khusus dari agregasi dengan hubungan yang paling kuat. Bagian (*part*) nggak bisa ada tanpa keseluruhannya (*whole*), jadi kalau *whole* hilang, *part*-nya ikut hilang. Contohnya Komputer yang punya Processor, Monitor, dan Mouse, dan semuanya dibuat langsung di dalam constructor Komputer.

B. Guided

1. 5.2.1 Guided1- Asosiasi

```
#Mahasiswa
package com.maysa.projectmodul5.Guided1.Asosiasi;

public class Mahasiswa {

    private String nim;
    private String nama;
    private int nilai;

    //Setter
    public void setNama(String nama) {
        this.nama = nama;
    }

    public void setNim(String nim) {
        this.nim = nim;
```

```
    }

    public void setNilai(int nilai) {
        this.nilai = nilai;
    }

    //Getter
    public int getNilai(){
        return this.nilai;
    }

    public String getNama() {
        return this.nama;
    }

    public String getNim() {
        return this.nim;
    }

}
```

#Dosen

```
package com.maysa.projectmodul5.Guided1.Asosiasi;

public class Dosen {

    private String kodeDosen;
    private String namaDosen;

    //Setter
    public void setKodeDosen(String kodeDosen) {
        this.kodeDosen = kodeDosen;
    }

    public void setNamaDosen(String namaDosen) {
        this.namaDosen = namaDosen;
    }

    //Getter
    public String getKodeDosen() {

        return this.kodeDosen;
    }

    public String getNamaDosen() {
        return this.namaDosen;
    }

    public void giveScore(Mahasiswa s, int nilai) {
    s.setNilai(nilai);
    }

    public int getScore(Mahasiswa s) {
```

```
        return s.getNilai();
    }

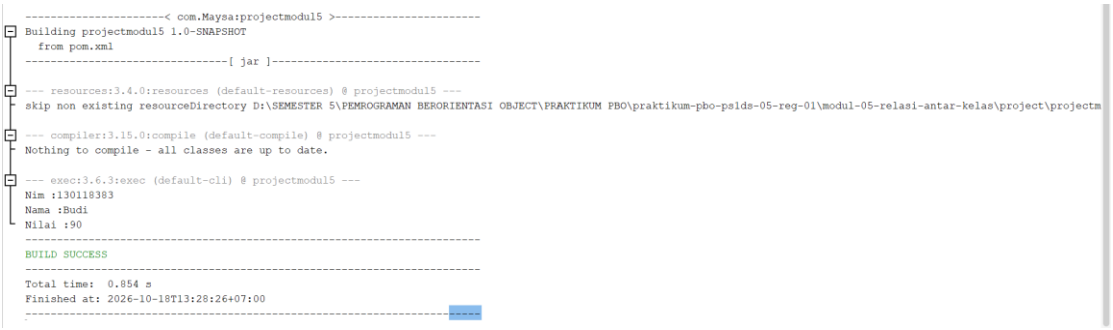
    public static void main(String[] args) {
        Mahasiswa m = new Mahasiswa();

        m.setNim("130118383");
        m.setNama("Budi");

        Dosen d = new Dosen();
        d.giveScore(m, 90);

        System.out.println("Nim :" + m.getNim());
        System.out.println("Nama :" + m.getNama());
        System.out.println("Nilai :" + d.getScore(m));
    }
}
```

Output :



Kesimpulan:

Dosen nggak menyimpan objek Mahasiswa sebagai atribut. Objeknya cuma dikirim lewat parameter di giveScore() dan getScore(). Nilai 90 tersimpan di objek m milik Mahasiswa, bukan di Dosen. Ini ciri asosiasi: dua kelas saling pakai tapi nggak saling memiliki.

2. 5.2.2 Agregasi

```
#Mahasiswa
package com.maysa.projectmodul5.Guided2.agregasi;

public class Mahasiswa {
    private String NIM, Nama;

    //constructor
    public Mahasiswa (String NIM, String Nama){
        this.NIM = NIM;
        this.Nama = Nama;
    }

    //Getter
    public String getNIM(){
        return NIM;
```

```
    }

    public String getNama(){
        return Nama;
    }

    //Setter
    public void setNIM(String NIM){
        this.NIM = NIM;
    }

    public void setNama(String Nama){
        this.Nama = Nama;
    }
}
```

#Jurusan

```
package com.maysa.projectmodul5.Guided2.agregasi;

public class Jurusan {
    //artinya milik itu sendiri
    private static int jumlahMahasiswa;
    private String kodeJurusan, namaJurusan;
    private Mahasiswa[] daftarMahasiswa = new Mahasiswa[32];

    //constructor
    public Jurusan (String kodeJurusan, String namaJurusan){
        this.kodeJurusan = kodeJurusan;
        this.namaJurusan = namaJurusan;
    }

    //Behaviour
    public void addMahasiswa(Mahasiswa m){
        this.daftarMahasiswa[jumlahMahasiswa]= m;
        jumlahMahasiswa++;
    }

    //menampilkan
    public void displayMahasiswa(){
        System.out.println("Kode Jurusan: " + this.kodeJurusan);
        System.out.println("Nama Jurusan: " + this.namaJurusan);

        System.out.println("Daftar Mahasiswa: ");

        for (int i = 0; i<jumlahMahasiswa;i++){
            System.out.println(i+1 + ". " + daftarMahasiswa[i].getNIM() + " " +
daftarMahasiswa[i].getNama());
        }
    }

    //Getter
    public String getKodeJurusan(){
        return kodeJurusan;
```

```
    }

  public String getNamaJurusan(){
     return namaJurusan;
  }
    //Setter

  public void setKodeJurusan(String kodeJurusan){
     this.kodeJurusan = kodeJurusan;
  }

  public void setNamaJurusan(String namaJurusan){
     this.namaJurusan = namaJurusan;
  }
}
```

3. 5.2.3 Komposisi

```
#Komputer
package com.maysa.projectmodul5.Guided2.komposisi;

public class Komputer {
    private Processor processor;
    private Monitor monitor;
    private Mouse mouse;

    public Komputer(String brandCpu, int kecepatanCpu, String brandMonitor, String
brandMouse, String tipeMouse){
        this.processor = new Processor(brandCpu, kecepatanCpu);
        this.monitor = new Monitor(brandMonitor);
        this.mouse = new Mouse(brandMouse, tipeMouse);
    }

    public void displaySpecKomputer() {
        System.out.println("=== SPESIFIKASI KOMPUTER ===");
        this.processor.displaySpec();
        this.monitor.DisplaySpec();
        this.mouse.displaySpec();
    }
}
```

```
#Monitor
package com.maysa.projectmodul5.Guided2.komposisi;

public class Monitor {
    private String brand;

    public Monitor(String brand){
        this.brand = brand;
    }

    public void DisplaySpec() {
        System.out.println(this.brand);
```

```
    }
}
```

```
#Mouse
package com.maysa.projectmodul5.Guided2.komposisi;

public class Mouse {
    private String brand, tipe;

    public Mouse(String brand, String tipe){
        this.brand = brand;
        this.tipe = tipe;
    }

    public void displaySpec(){
        System.out.println(this.brand + ", " + this.tipe);
    }
}
```

```
#Processor
package com.maysa.projectmodul5.Guided2.komposisi;

public class Processor {
    private String brand;
    private int kecepatan;

    //constructor
    public Processor(String brand,int kecepatan){
        this.brand = brand;
        this.kecepatan = kecepatan;
    }

    //behaviour
    public void displaySpec(){
        System.out.println(this.brand + ", " + this.kecepatan);
    }
}
```

Main

```
#Main
package com.maysa.projectmodul5;
import com.maysa.projectmodul5.Guided2.komposisi.Komputer;
import com.maysa.projectmodul5.Guided2.agregasi.*;

public class Main {
    public static void main(String[] args) {
        runGuided2();
        runGuided3();
    }


    public static void runGuided1(){}
```

```
    public static void runGuided2(){
        Mahasiswa m1 = new Mahasiswa("103112400148", "Rendhi");
        Mahasiswa m2 = new Mahasiswa("103112400149", "John");

        Jurusan prodi = new Jurusan("SD","Sains Data");

        prodi.addMahasiswa(m1);
        prodi.addMahasiswa(m2);

        prodi.displayMahasiswa();

    }

    public static void runGuided3(){
        Komputer pc = new Komputer("Intel Core i5",3200,"LG 24
Inch","Logitech","Wireless");
        pc.displaySpecKomputer();
    }
}
```

Output:



Kesimpulan:

- Agregasi (Jurusan): objek m1 dan m2 dibuat dulu di main, lalu dimasukkan ke Jurusan lewat addMahasiswa(). Jadi Mahasiswa bisa berdiri sendiri tanpa Jurusan, dan itu yang bikin hubungannya agregasi.
- Komposisi (Komputer): objek Processor, Monitor, dan Mouse nggak dibuat di main, tapi langsung di dalam constructor Komputer lewat new. Di main kita cuma bikin satu objek Komputer, dan ketiga komponennya ikut terbentuk. Kalau Komputer hilang, komponennya ikut hilang. Itu ciri komposisi.

C. Unguided

Soal Unguided — Modul 5: Pemrograman Berorientasi Objek

Relasi Antar Kelas: Komposisi, Agregasi, dan Asosiasi

Petunjuk Umum

Buatlah program Java berdasarkan cerita berikut. Rancang sendiri struktur program yang diperlukan sebelum menuliskan kode.

- Tentukan sendiri field, constructor, dan method yang diperlukan, mengikuti hint dan tugas yang disediakan.
- Data di dalam object tidak boleh diakses atau diubah langsung dari luar class-nya (gunakan enkapsulasi).
- Gunakan nama parameter yang sama dengan nama field pada constructor (sehingga this perlu dipakai secara eksplisit).
- Proyek wajib mengikuti konvensi penamaan standar (PascalCase untuk Class, camelCase untuk variabel/method).
- Gunakan array standar bertipe tetap (bukan ArrayList atau koleksi library lainnya).
- Tidak perlu menggunakan inheritance, overriding, maupun abstract class.

Soal — Sistem Manajemen Pesanan Cafe

Sebuah cafe membutuhkan sistem sederhana untuk mencatat dan memproses pesanan pelanggan. Dalam sistem ini, relasi antar kelas dirancang untuk mencerminkan kondisi nyata operasional cafe.

Pengerjaan soal ini dibagi menjadi 3 bagian tugas utama yang masing-masing melatih satu jenis relasi antar kelas, diikuti dengan pengujian pada kelas utama (Main).

Tugas 1: Implementasi Komposisi (Composition)

Komposisi adalah relasi di mana sebuah objek memiliki objek lain sebagai bagian internalnya yang **tidak dapat berdiri sendiri**. Siklus hidup objek internal terikat penuh pada objek utamanya.

Pada sistem cafe:

- Setiap item pesanan (ItemPesanan) pasti memiliki rincian harga (RincianHarga) yang mencatat harga dasar dan pajak (10%).

- Objek RincianHarga harus di-instansiasi langsung di dalam constructor ItemPesanan. Objek RincianHarga tidak pernah dibuat di main. Jika ItemPesanan dihapus, maka RincianHarga miliknya ikut hilang.

Kebutuhan Tugas 1

Buatlah dua class: RincianHarga dan ItemPesanan.

Field yang disarankan:

Class	Field	Jenis	Tipe	Keterangan
RincianHarga	hargaDasar	Instance field	double	Harga awal sebelum pajak
RincianHarga	pajak	Instance field	double	Nilai nominal pajak (10% dari harga dasar)
ItemPesanan	namaItem	Instance field	String	Nama menu (misal: "Iced Americano")
ItemPesanan	rincian	Instance field	RincianHarga	Objek internal rincian harga (**Komposisi**)

Method yang disarankan:

Class	Method	Return	Keterangan
RincianHarga	getHargaTotal()	double	Mengembalikan nilai hargaDasar + pajak
ItemPesanan	getNamaItem()	String	Mengembalikan nama item pesanan
ItemPesanan	getRincian()	RincianHarga	Mengembalikan objek rincian harga

---

Tugas 2: Implementasi Agregasi (Aggregation)

Agregasi adalah relasi di mana sebuah objek menampung kumpulan objek lain yang **dapat berdiri sendiri**. Objek yang ditampung dibuat secara independen di luar, lalu dimasukkan ke dalam objek wadah.

Pada sistem cafe:

- Sebuah Pesanan menampung beberapa objek ItemPesanan menggunakan array standar.

- Objek ItemPesanan dibuat terpisah terlebih dahulu (di main), lalu dimasukkan satu per satu ke dalam Pesanan melalui method tambahItem(). Jika Pesanan dibatalkan atau dihapus, objek ItemPesanan tersebut secara konsep tetap ada.

Kebutuhan Tugas 2

Buatlah class Pesanan.

Field yang disarankan:

Field	Jenis	Tipe	Keterangan
nomorMeja	Instance field	int	Nomor meja pelanggan
daftarItem	Instance field	ItemPesanan[]	Array penampung item pesanan (**Agregasi**)
jumlahItem	Instance field	int	Penanda jumlah item yang sudah masuk ke array

Method yang disarankan:

Method	Return	Keterangan
tambahItem(ItemPesanan item)	void	Memasukkan objek ItemPesanan dari luar ke dalam array daftarItem
tampilkanPesanan()	void	Menampilkan nomor meja dan seluruh item beserta harga total per item
getNomorMeja()	int	Mengembalikan nomor meja
getDaftarItem()	ItemPesanan[]	Mengembalikan array daftar item
getJumlahItem()	int	Mengembalikan jumlah item yang tersimpan

Tugas 3: Implementasi Asosiasi (Association)

Asosiasi adalah relasi interaksi antara dua objek independen yang **saling menggunakan tetapi tidak saling memiliki**. Objek tidak menyimpan objek lain sebagai field permanen, melainkan berinteraksi melalui parameter method.

Pada sistem cafe:

- Seorang Kasir memproses pembayaran dari sebuah Pesanan.
- Class Kasir tidak menyimpan objek Pesanan sebagai field, melainkan menerima objek Pesanan sebagai parameter pada method prosesPembayaran().

Kebutuhan Tugas 3

Buatlah class Kasir.

Field yang disarankan:

Field	Jenis	Tipe	Keterangan
namaKasir	Instance field	String	Nama petugas kasir

Method yang disarankan:

Method	Return	Keterangan
prosesPembayaran(Pesanan pesanan)	void	Mengakses data Pesanan, menghitung akumulasi total bayar seluruh item, lalu mencetak struk pembayaran (**Asosiasi**)

Tugas 4: Eksekusi dan Pengujian di Main
Buatlah class Main untuk merangkai seluruh kelas dan menguji ketiga jenis relasi.

Alur program di main

1. Buat 1 objek Kasir (misal: "Budi").
2. Buat 3 objek ItemPesanan secara independen:
   - "Iced Americano", harga dasar: 18000
   - "Beef Croissant", harga dasar: 25000
   - "Matcha Latte", harga dasar: 22000
3. Buat 1 objek Pesanan untuk **Meja 4** (kapasitas array misal: 5 item).
4. Masukkan ketiga objek ItemPesanan ke dalam objek Pesanan menggunakan method tambahItem().
5. Tampilkan rincian pesanan dengan memanggil tampilkanPesanan() pada objek Pesanan.
6. Jalankan pemrosesan pembayaran dengan memanggil method prosesPembayaran(pesanan) dari objek Kasir.

Output yang Diharapkan
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4

```
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
==============================
```

Yang Perlu Diperhatikan Saat Mengerjakan

- **Komposisi:** Objek RincianHarga wajib di-instansiasi di dalam constructor ItemPesanan menggunakan kata kunci new. Dilarang melakukan new RincianHarga(...) di dalam main.
- **Agregasi:** Objek ItemPesanan wajib di-instansiasi di main terlebih dahulu, kemudian dikirimkan sebagai argumen ke method tambahItem().
- **Asosiasi:** Class Kasir dilarang memiliki field bertipe Pesanan. Interaksi hanya terjadi saat method prosesPembayaran(Pesanan pesanan) dipanggil.

Jawaban :

Unguided-Agregasi(Pesanan), Asosiasi(Kasir), Komposisi(ItemPesanan, RincianHarga), Unguided(Main)

1. Unguided-Agregasi

```
#Pesanan
package com.maysa.projectmodul5.Unguided.Agregasi;
import com.maysa.projectmodul5.Unguided.Komposisi.ItemPesanan;
import com.maysa.projectmodul5.Unguided.Komposisi.RincianHarga;

public class Pesanan {
    private int nomorMeja;
    private ItemPesanan[] daftarItem; // Agregasi
    private int jumlahItem;

    public Pesanan(int nomorMeja, int kapasitas) {
        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[kapasitas];
        this.jumlahItem = 0;
    }

    public void tambahItem(ItemPesanan item) {
        if (this.jumlahItem < this.daftarItem.length) {
            this.daftarItem[this.jumlahItem] = item;
```

```
            this.jumlahItem++;
        } else {
            System.out.println("Pesanan penuh, item " + item.getNamaItem() + " tidak dapat
ditambahkan.");
        }
    }

    public void tampilkanPesanan() {
        System.out.println("=== DAFTAR PESANAN CAFE ===");
        System.out.println("Nomor Meja: " + this.nomorMeja);
        for (int i = 0; i < this.jumlahItem; i++) {
            ItemPesanan item = this.daftarItem[i];
            RincianHarga r = item.getRincian();
            System.out.println((i + 1) + ". " + item.getNamaItem()
                    + " - Rp " + r.getHargaTotal()
                    + " (Harga: " + r.getHargaDasar()
                    + ", Pajak: " + r.getPajak() + ")");
        }
        System.out.println();
    }

    public int getNomorMeja() {
        return this.nomorMeja;
    }

    public ItemPesanan[] getDaftarItem() {
        return this.daftarItem;
    }

    public int getJumlahItem() {
        return this.jumlahItem;
    }
}
```

2. Unguided-Asosiasi

```
#Kasir

package com.maysa.projectmodul5.Unguided.Asosiasi;

import com.maysa.projectmodul5.Unguided.Agregasi.Pesanan;
import com.maysa.projectmodul5.Unguided.Komposisi.ItemPesanan;
import com.maysa.projectmodul5.Unguided.Komposisi.RincianHarga;

public class Kasir {
    private String namaKasir;

    public Kasir(String namaKasir) {
        this.namaKasir = namaKasir;
    }

    // Asosiasi: Pesanan hanya diterima sebagai parameter, bukan disimpan sebagai field
    public void prosesPembayaran(Pesanan pesanan) {
```

```
        double totalSubtotal = 0;
        double totalPajak = 0;
        ItemPesanan[] items = pesanan.getDaftarItem();

        for (int i = 0; i < pesanan.getJumlahItem(); i++) {
            RincianHarga r = items[i].getRincian();
            totalSubtotal += r.getHargaDasar();
            totalPajak += r.getPajak();
        }
        double totalBayar = totalSubtotal + totalPajak;

        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + this.namaKasir);
        System.out.println("Pesanan        : Meja " + pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : " + pesanan.getJumlahItem());
        System.out.println("Total Subtotal : Rp " + totalSubtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp " + totalBayar);
        System.out.println("==============================");
    }
}
```

3. Unguided-Komposisi

```
#ItemPesanan
package com.maysa.projectmodul5.Unguided.Komposisi;

public class ItemPesanan {
    private String namaItem;
    private RincianHarga rincian; // Komposisi

    public ItemPesanan(String namaItem, double hargaDasar) {
        this.namaItem = namaItem;
        // Komposisi: RincianHarga dibuat di dalam constructor ItemPesanan
        this.rincian = new RincianHarga(hargaDasar);
    }

    public String getNamaItem() {
        return this.namaItem;
    }

    public RincianHarga getRincian() {
        return this.rincian;
    }
}
```

```
#RincianHarga
package com.maysa.projectmodul5.Unguided.Komposisi;

public class RincianHarga {
    private double hargaDasar;
    private double pajak;
```

```
    public RincianHarga(double hargaDasar) {
        this.hargaDasar = hargaDasar;
        this.pajak = hargaDasar * 10 / 100; // pajak 10% dari harga dasar
    }

    public double getHargaDasar() {
        return this.hargaDasar;
    }

    public double getPajak() {
        return this.pajak;
    }

    public double getHargaTotal() {
        return this.hargaDasar + this.pajak;
    }
}
```

4. Unguided-Main

```
#Main
package com.maysa.projectmodul5.Unguided;

import com.maysa.projectmodul5.Unguided.Agregasi.Pesanan;
import com.maysa.projectmodul5.Unguided.Asosiasi.Kasir;
import com.maysa.projectmodul5.Unguided.Komposisi.ItemPesanan;

public class Main {
    public static void main(String[] args) {
        // 1. Kasir
        Kasir kasir = new Kasir("Budi");

        // 2. ItemPesanan dibuat independen di main
        //    (RincianHarga dibuat otomatis di dalam ItemPesanan)
        ItemPesanan item1 = new ItemPesanan("Iced Americano", 18000);
        ItemPesanan item2 = new ItemPesanan("Beef Croissant", 25000);
        ItemPesanan item3 = new ItemPesanan("Matcha Latte", 22000);

        // 3. Pesanan untuk Meja 4, kapasitas 5 item
        Pesanan pesanan = new Pesanan(4, 5);

        // 4. Agregasi: memasukkan item yang sudah ada ke dalam pesanan
        pesanan.tambahItem(item1);
        pesanan.tambahItem(item2);
        pesanan.tambahItem(item3);

        // 5. Tampilkan pesanan
        pesanan.tampilkanPesanan();

        // 6. Asosiasi: kasir memproses pembayaran
        kasir.prosesPembayaran(pesanan);
    }
```

```
}
```

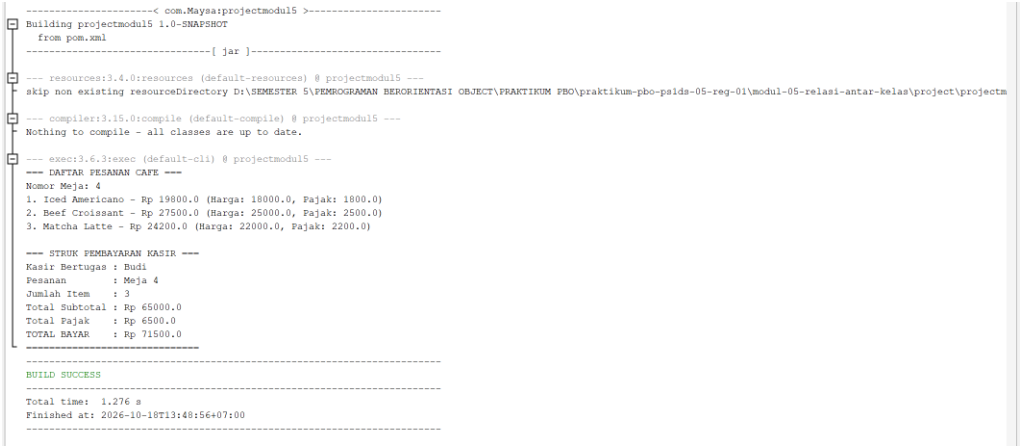
Penjelasan source code:

Program ini mensimulasikan sistem pesanan cafe dengan tiga jenis relasi antar kelas.

1. Agregasi (Pesanan). Pesanan punya array daftarItem bertipe ItemPesanan[] dengan ukuran sesuai kapasitas, dan jumlahItem buat nandain berapa slot yang sudah terisi. Method tambahItem() menerima objek ItemPesanan dari luar dan memasukkannya ke array. Ada pengecekan kapasitas, jadi kalau array penuh akan muncul pesan dan item nggak ditambahkan. tampilkanPesanan() mencetak nomor meja dan tiap item beserta harga total, harga dasar, dan pajaknya. ItemPesanan tetap ada walaupun Pesanan dihapus, karena dibuat terpisah di main.
2. Asosiasi (Kasir). Kasir cuma punya field namaKasir. Objek Pesanan nggak disimpan sebagai field, tapi diterima sebagai parameter di prosesPembayaran(Pesanan pesanan). Di method itu, kasir mengambil daftar item lewat getter, menjumlahkan harga dasar dan pajak seluruh item lewat perulangan for, lalu mencetak struk.
3. Komposisi (ItemPesanan dan RincianHarga). RincianHarga menyimpan hargaDasar dan pajak, yang dihitung otomatis 10% dari harga dasar di constructor-nya. getHargaTotal() mengembalikan harga dasar + pajak. ItemPesanan menyimpan namaItem dan rincian. Objek RincianHarga dibuat pakai new di dalam constructor ItemPesanan, bukan di main, jadi RincianHarga nggak bisa berdiri sendiri dan hidupnya terikat ke ItemPesanan.
4. Main. Pertama dibuat Kasir bernama Budi, lalu 3 objek ItemPesanan secara independen, lalu Pesanan untuk Meja 4 berkapasitas 5. Ketiga item dimasukkan lewat tambahItem(), kemudian tampilkanPesanan() dan prosesPembayaran(pesanan) dipanggil.

Semua atribut dibuat private dan diakses lewat getter (enkapsulasi), dan constructor memakai this karena nama parameter sama dengan nama field.

Output:



Penjelasan Output:
Bagian pertama keluar dari tampilkanPesanan() di Pesanan. Pajak tiap item dihitung 10% dari harga dasar: Iced Americano 18000 jadi pajak 1800 dan total 19800, Beef Croissant 25000 jadi pajak 2500 dan total 27500, Matcha Latte 22000 jadi pajak 2200 dan total 24200. Bagian kedua keluar dari prosesPembayaran() di Kasir. Total subtotal adalah 18000 + 25000 + 22000 = 65000, total pajak 1800 + 2500 + 2200 = 6500, dan total bayar 65000 + 6500 = 71500. Hasilnya sama persis dengan output yang diminta di soal.

D. Kesimpulan

Dari bagian guided, perbedaan ketiga relasi antar kelas terlihat dari cara objek dibuat dan disimpan. Pada asosiasi, Dosen tidak menyimpan objek Mahasiswa dan hanya menerimanya lewat parameter giveScore() dan getScore(), sehingga nilai 90 tetap tersimpan di objek Mahasiswa. Pada agregasi, objek Mahasiswa dibuat terlebih dahulu di main lalu dimasukkan ke array milik Jurusan lewat addMahasiswa(), sehingga Mahasiswa tetap bisa ada tanpa Jurusan dan outputnya menampilkan dua mahasiswa dalam satu jurusan. Pada komposisi, Processor, Monitor, dan Mouse dibuat langsung di dalam constructor Komputer, sehingga ketiganya tidak bisa berdiri sendiri dan terikat pada Komputer.

Pada bagian unguided, ketiga relasi diterapkan sekaligus dalam sistem pesanan cafe. RincianHarga dibuat di dalam constructor ItemPesanan dengan pajak 10% yang dihitung otomatis (komposisi). Tiga objek ItemPesanan dibuat di main lalu dimasukkan ke Pesanan lewat tambahItem(), dengan pengecekan kapasitas array agar tidak melebihi batas (agregasi). Kasir hanya menerima Pesanan sebagai parameter prosesPembayaran() tanpa menyimpannya sebagai field (asosiasi). Hasilnya sesuai dengan yang diminta soal, yaitu pesanan Meja 4 dengan 3 item, total subtotal Rp 65000.0, total pajak Rp 6500.0, dan total bayar Rp 71500.0. Seluruh atribut dibuat private dan diakses lewat getter, sehingga data objek tidak bisa diubah langsung dari luar kelasnya.

Dari keseluruhan praktikum, dapat disimpulkan bahwa komposisi digunakan ketika suatu objek merupakan bagian yang tidak terpisahkan dari objek utamanya, agregasi ketika objek yang ditampung masih bisa berdiri sendiri, dan asosiasi ketika dua objek hanya saling menggunakan lewat method. Dengan begitu, tujuan praktikum untuk memahami hubungan antar kelas dan menerapkannya dalam program Java sudah tercapai.